15/10/25

1. Compiler:

* It translates the entire source program into machine code or another lower-level form before execution.
* It analyses the complete program
* The compiled program can usually execute faster because translation has already been performed.

Eg.: C & C++ use GCC compilers.

Advantages:

* Faster execution
* Many errors can be detected before the program runs.
* The resulting executable can run without recompiling every time.

Disadvantages:

* Compilation takes more time
* Errors may need to be fixed before the program can be executed successfully.

2. Interpreter:

* An interpreter is a program that translates and executes source code line by line at runtime.
* It reads one statement at a time
* It executes the statement immediately.
* If an error occurs, execution usually stops at that point.
* It does not normally create a separate executable file before running the program. Eg: Python.

Advantages:

* Easy to test & debug.
* Errors can be identified quickly
* No separate compilation step is usually Required

Disadvantages:

* Execution can be slower because code is processed at runtime.
* The source code generally needs an interpreter to run.

	Interpreter	Compiler
i)	Translates & executes during runtime	Translates before execution
ii)	Usually processes code incrementally	Usually processes the program as a whole.
iii)	Generally slower execution	Faster execution
iv)	Easy for interactive testing	Better suited for optimized executable programs.
v)	Python interpreter	C/C++ compiler.

2. OOPs:

+ object-Oriented programming is a programming approach in which a program is designed using objects & classes.

+ An object represents i) Data → properties / attributes
ii) Behaviour → methods / functions.

Class:

+ It is a blueprint or template used to create objects.

Object: It is an instance of class.

Main principles of oop:

i) **Encapsulation**:
Combining data & methods into a single unit, usually a class & controlling access to the data.

ii) **Inheritance**:
It allows one class to acquire properties & methods from another class.

iii) **polymorphism**:
One interface or method name can have different implementations.

Eg: Different classes can have the same `sound()` method but produce different sounds.

iv) **Abstraction**:
Hiding unnecessary implementation details and showing only essential features to the user.

Eg: Driver uses steering wheel & Pedals without needing to know the internal of the Mechanism.

Advantages:

* provides better code organization
* Promotes code reusability.
* Makes larger programs easier to maintain.
* provides data security through encapsulation
* Makes programs easier to modify & extend.
* Models real world entities effectively.